

RoboLongR: Causally-Consistent Rewards Teach Robots to Progressively Master Long-Horizon Manipulation

Anonymous RA-L Submission

Abstract—Learning rewards for long-horizon manipulation requires distinguishing task completion from visually similar states reached without satisfying task prerequisites. We propose **RoboLongR**, an offline-to-online framework targeting causally consistent rewards that assess progress against task prerequisites using execution history. Stage-shuffle supervision and recurrent memory encourage execution-order sensitivity, while two-round reward learning refines trajectory segmentation. The learned rewards guide stage-conditioned policy refinement from behavior cloning to online reinforcement learning, with sparse terminal feedback triggering reward correction for repeated high-reward failures. Pretrained on demonstrations from 88 RL Bench tasks, RoboLongR achieves 31.8% average success on 18 long-horizon tasks excluded from reward pretraining, exceeding the strongest evaluated baseline by 9.2 percentage points. On four real-robot tasks, it achieves 57.5% success, a 32.5-percentage-point improvement over the comparison method. Reward-level controls show sensitivity to synthetic order perturbations, while ablations examine component contributions to downstream policy improvement. Anonymous project website: <https://Robo-LongR.github.io>.

Index Terms—Long-horizon manipulation, causally consistent rewards, reinforcement learning, learning from demonstration.

I. INTRODUCTION

Long-horizon manipulation requires a robot to coordinate actions whose validity depends on earlier interactions. To store an object in a drawer, the robot must open the drawer, place the object inside, and close it. Closing an empty drawer produces a goal-like image without completing the task. As shown in Fig. 1, a reward based on apparent progress can therefore reinforce behavior that skips essential prerequisites. This ambiguity becomes particularly consequential when online learning takes the policy beyond its demonstrations [1], [2]. Demonstrations initialize robot behavior, while dense visual rewards support subsequent refinement through interaction [3]–[7]. However, recognizing a later task state does not establish whether the robot reached it through a valid execution. Causally consistent rewards should account for the execution history that supports task progress. This motivates our central question: *How can demonstrations and sparse interaction feedback teach causally consistent rewards that distinguish apparent progress from progress supported by task prerequisites?*

The distinction is temporal as well as visual. Two observations can show the same closed drawer while corresponding to different task outcomes, depending on whether insertion occurred earlier. Conversely, an unsuccessful intermediate attempt may be followed by a valid recovery. A useful reward should retain evidence about prior interactions while

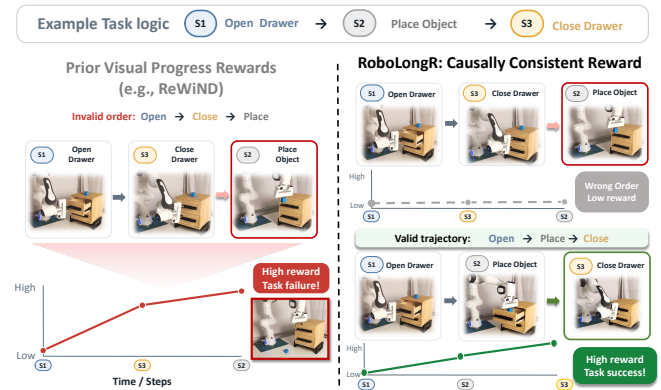


Fig. 1. **Beyond visual progress: learning causally consistent rewards.** A closed drawer may resemble the goal even when the object is absent. Stage-shuffle supervises execution-order sensitivity as a proxy for prerequisite sensitivity in history-dependent rewards. Curves schematically illustrate the intended reward behavior, with real-execution evaluation reported separately.

remaining calibrated as the policy visits new states. This creates two related challenges: learning order sensitivity from successful demonstrations, which contain few explicit failure examples, and maintaining that sensitivity under the distribution of online policy rollouts.

We address this question with **RoboLongR**, an offline-to-online framework that connects causally consistent reward learning with progressive policy refinement. We define *causal consistency* as compatibility between rewarded progress and the task-defined prerequisites supported by the execution history. These prerequisite relations may form a partial order and admit several valid executions. We pursue this objective through order supervision, using perturbation controls and downstream success for evaluation. RoboLongR has three components. First, *stage-shuffle* reorders intact demonstration segments, providing order supervision as a proxy for causal consistency. Recurrent memory retains observations across temporal windows, and a second training round adapts the segments to predicted progress. Second, separate coarse stages condition behavior cloning (BC) and a scheduled reinforcement-learning (RL) phase. All RL episodes start from ordinary initial states, keeping complete-task execution as the objective. Third, sparse terminal feedback identifies repeated failures with high predicted rewards and triggers reward correction. This closes the loop between policy experience and the reward used for further learning.

We pretrain the reward model on demonstrations from 88 RL Bench tasks and evaluate policy improvement on 18 distinct long-horizon tasks excluded from offline reward pretraining. Downstream learning uses task demonstrations

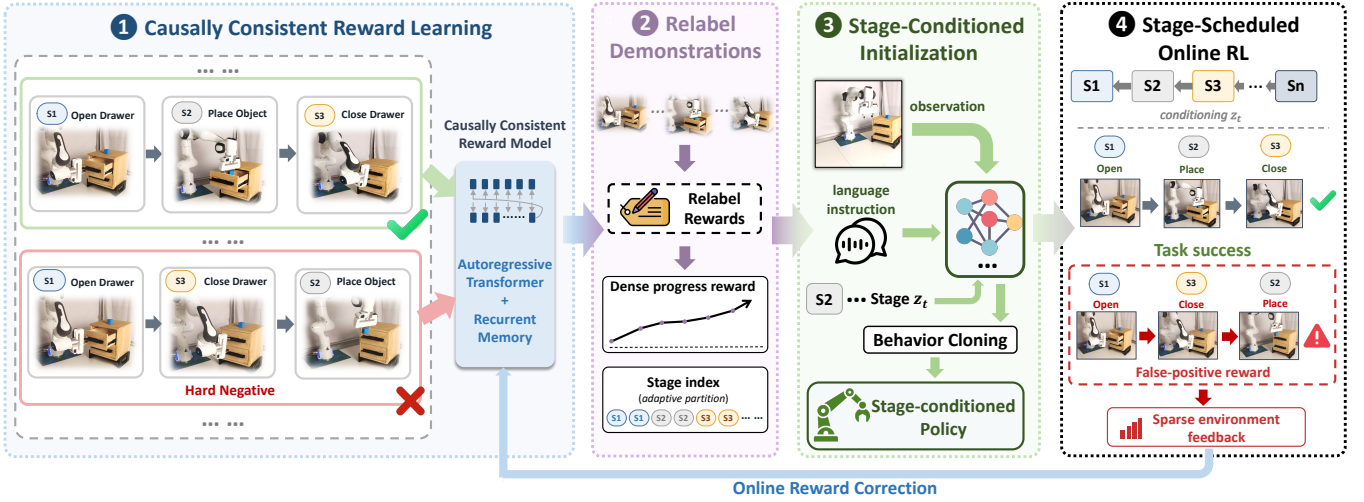


Fig. 2. **RoboLongR targets causally consistent rewards for progressive policy refinement.** Stage-shuffle supplies order supervision using reward-adaptive segments. Separate coarse stages condition BC and scheduled online RL. Sparse terminal feedback flags repeated high-scoring failures for reward correction, connecting policy interaction to subsequent reward updates.

for BC initialization and interaction feedback for policy refinement and reward correction. RoboLongR achieves 31.8% average success in simulation and 57.5% across four real-robot tasks, improving over the comparison baseline by 9.2 and 32.5 percentage points, respectively. In summary, our contributions are: (1) stage-order supervision and recurrent memory for progress rewards targeting prerequisite sensitivity; (2) a BC-to-RL framework that separates reward segmentation from policy conditioning and corrects rewards using interaction feedback; and (3) evaluations on tasks held out from reward pretraining and on real robots, examining policy performance and learned order sensitivity.

II. RELATED WORK

Rewards for Robot Learning. Learning rewards from demonstrations has long been studied through inverse reinforcement learning and adversarial imitation learning [8], [9]. Recent approaches exploit visual and language representations to provide progress, value, or similarity signals for robot control, including LIV [3], VLC [10], GVL [11], and RoboCLIP [4]. ReWiND [5] uses rewound trajectories to learn progress rewards for offline-to-online reinforcement learning, while general-purpose reward models explore trajectory comparisons and process supervision [12]–[14]. These advances provide increasingly informative learning signals, but estimating progress alone does not ensure that earlier task prerequisites have been satisfied. RoboLongR targets causal consistency through stage-order supervision and history-dependent reward learning, aiming to make progress estimates sensitive to task prerequisites.

Stage-Aware Rewards and Self-Improvement. SARM [15] and SARM2 [16] explicitly model task stages and progress, demonstrating the relevance of stage-aware rewards to long-horizon manipulation and robot self-improvement. Confidence gating, as explored in RARM [17], offers a complementary mechanism for handling unreliable reward estimates. Meanwhile, interaction-based policy refinement [6],

[7], [18]–[20] enables robots to improve through experience, motivating rewards that remain reliable as policy behavior evolves. Building on these directions, RoboLongR combines shuffled-stage supervision with stage-conditioned policy refinement in a BC-to-RL pipeline. Sparse terminal feedback further identifies repeated high-reward failures and triggers online reward correction, adapting reward estimates to failures encountered during policy execution.

III. METHOD

Fig. 2 presents an overview of RoboLongR, combining causally consistent reward learning with online policy refinement. We first learn a history-dependent reward from demonstrations, then use it for stage-conditioned policy refinement and correct reward estimates for high-scoring failures during interaction.

A. Problem Formulation and Reward Representation

A task instruction ℓ specifies trajectories $\tau = \{(o_t, s_t, a_t)\}_{t=1}^T$, where o_t , s_t , and a_t denote RGB observations, robot state, and actions. The environment supplies binary terminal success $b(\tau)$. We learn a dense progress reward $R_\phi(o_{1:T}, \ell) \in [0, 1]$ to optimize a policy π_θ .

Task prerequisites define which execution orders are valid. Our causal-consistency objective is to reward progress when the observed history supports the prerequisites of the current task state. Independent placements may be exchanged, whereas placing an arch requires both supports. We do not observe this semantic prerequisite graph. Instead, contiguous demonstration segments $S_1, \dots, S_{K_a(\tau)}$ provide units for reward augmentation. A separate coarse stage $z_t \in \{1, \dots, K_c\}$ conditions the policy. Adaptive segments need not coincide with semantic subtasks; coarse stages provide a consistent policy input across demonstrations.

The reward uses two frozen encoders: DINOv2 ViT-B/14 [21] for two RGB views and all-MiniLM-L6-v2 for language. Mean-pooled visual features and language features

are projected to dimension 512 and processed by a four-layer, eight-head autoregressive Transformer. A recurrent module of hidden size 512 and an MLP predict progress. Attention spans 100 frames; longer sequences use causal chunks with recurrent state carried between chunks, detached during training, and reset at episode boundaries. Thus each prediction depends only on the observed prefix while retaining information from earlier windows.

The two visual streams yield 768-dimensional features, and the language encoder yields a 384-dimensional embedding. Images use 224×224 crops with DINOv2 normalization; the Transformer uses a feed-forward dimension of 2048 and dropout 0.1. Freezing the encoders keeps the representation function fixed during offline reward learning and online correction. The trainable temporal modules can therefore revise how observation sequences are scored without changing the visual and language backbones.

History is important because a single frame may not reveal whether an occluded prerequisite has been completed. Recurrent state provides a mechanism for carrying this evidence beyond the finite attention window. Stage-shuffle supplies the complementary learning signal: histories containing the same intact segments receive different targets when their execution order changes. History dependence makes earlier observations available, while stage-order supervision encourages sensitivity to their ordering. Together they support the causal-consistency objective without explicitly verifying prerequisite satisfaction.

B. Stage-Order Supervision for Causal Consistency

We use stage-order supervision as a training proxy for causal consistency. For three consecutive demonstration segments, stage-shuffle replaces $S_i \rightarrow S_{i+1} \rightarrow S_{i+2}$ with $S_i \rightarrow S_{i+2} \rightarrow S_{i+1}$. Segment contents remain intact, while their order and boundary transitions change. Let t_* denote the first displaced step. The augmented target preserves the positive target before t_* and becomes zero thereafter:

$$\tilde{y}_t = \begin{cases} y_t, & t < t_*, \\ 0, & t \geq t_*. \end{cases} \quad (1)$$

A shuffled counterpart is generated with probability $p = 0.5$. For a positive trajectory and its augmented counterpart, the masked regression objective is:

$$\mathcal{L}_R = \frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} [(R_\phi(\tau, \ell)_t - y_t)^2 + (R_\phi(\tilde{\tau}, \ell)_t - \tilde{y}_t)^2], \quad (2)$$

where $\mathcal{M}$ excludes padding. The zero suffix is a training prior rather than a semantic validity label. It may penalize valid alternative orders or recoverable errors. To discourage indiscriminate cut detection, we also construct order-preserving positive splices by replacing a segment with the corresponding stage from another successful demonstration. Their local boundary changes are not guaranteed to match those of shuffled trajectories; reward-level controls examine this distinction in Sec. IV-C.

Keeping each segment intact preserves its within-stage visual progression. Compared with reversing a sequence,

the perturbation changes the ordering of locally plausible behaviors and targets a different ambiguity: a later-looking state may be reached before its supporting history. In drawer storage, for example, an opening–closing–placement permutation uses familiar actions while breaking the demonstrated ordering. This synthetic construction is inexpensive, but a temporal segment is not a certified semantic action. The positive-splice control and the distinction between physical task dependencies and temporal partitions are therefore essential to interpreting the learned reward.

C. Reward-Adaptive Segmentation

Reward training has two rounds (Algorithm 1). Round 1 uses uniform temporal segments and targets $y_t = (t/T)^2$. Round 2 continues from the first checkpoint, using its predicted curves as targets after per-demonstration min-max normalization. We smooth each curve with an 11-frame, degree-three Savitzky–Golay filter and select local maxima of the absolute second derivative above one standard deviation of curvature magnitude. Boundaries are separated by at least five frames; trajectories without a sufficiently strong change point use five uniform segments. Stage-shuffle requires at least three segments.

Adaptive partitions determine the reward-training perturbations. Policy stages instead follow task-category templates, such as reach, grasp, lift, transport, and place, with boundaries assigned from gripper transitions, object motion, contact, and release. These engineered cues keep the conditioning vocabulary consistent across demonstrations while allowing reward segments to vary with each trajectory.

The two representations serve different purposes. A change point in a reward curve may lie inside a semantic subtask, and a smooth interval may span several actions. Adaptive indices consequently vary across demonstrations and are useful for constructing trajectory-specific perturbations. Feeding those indices directly to the policy would also change the meaning of its conditioning input across examples. Coarse event-based labels retain a shared vocabulary at the cost of engineered supervision. The segmentation and stage-assignment ablations test this division of labor.

D. Progressive Policy Refinement

The actor and critic receive joint positions and velocities, end-effector pose, gripper opening, a frozen language embedding $\mathbf{e}_\ell$, and a learned stage embedding. Actions comprise an end-effector displacement, quaternion rotation, and gripper command. We initialize $\pi_\theta(s_t, \mathbf{e}_\ell, z_t)$ with 50 epochs of BC using per-step coarse labels. PPO [22] then optimizes the raw learned progress reward: $r_t^{\text{RL}} = R_\phi(o_{1:t}, \ell)$.

Terminal success is used for scheduling and reward correction, rather than added to this objective. During RL, the stage input stays fixed within an episode. It starts at the final stage and advances toward the first when success exceeds 0.8. Every episode starts from the ordinary initial-state distribution; no mid-task resets are used. Evaluation fixes the stage input to the first stage. Thus “progressive”

Algorithm 1 Two-round reward training with stage-shuffle

Require: demonstrations $\mathcal{D}$, instructions ℓ , shuffle rate p

- 1: initialize R_ϕ and set $y_t = (t/T)^2$
- 2: **for** $q = 1, 2$ **do**
- 3: **if** $q = 1$ **then**
- 4: partition each trajectory uniformly
- 5: **else**
- 6: predict and smooth progress, then compute adaptive partitions and targets
- 7: **end if**
- 8: **for** each minibatch $B \subset \mathcal{D}$ **do**
- 9: construct order-preserving positive splices
- 10: with probability p , propose a three-stage permutation
- 11: set the negative target to zero from the first displaced stage
- 12: update ϕ with masked positive and negative regression losses
- 13: **end for**
- 14: **end for**
- 15: **return** R_ϕ

describes reward refinement and the conditioning schedule, without assuming monotonic improvement at every update.

E. Online Reward Correction

Policy interaction can expose failed executions with high predicted progress. We flag a rollout when $b(\tau) = 0$ and $\max_t R_\phi(o_{1:t}, \ell) > 0.65$. Three consecutive flagged episodes trigger a task-specific correction: $\mathcal{L}_{\text{corr}} = \frac{1}{T} \sum_{t=1}^T R_\phi(\tau^-, \ell)_t^2$.

One Adam step at learning rate 10^{-4} updates the Transformer, recurrent module, and progress head, with gradient norm clipped to 1.0. Both encoders remain frozen. The counter is then reset, and no offline replay is included. This feedback requires no new per-step labels, but terminal failure cannot localize the error: whole-trajectory correction may also suppress valid prefixes.

The correction trigger treats repeated high predicted progress without task completion as a signal for reward revision. High progress on a failed rollout need not be erroneous, since valid partial progress can precede a late control failure. The trigger therefore identifies candidate reward errors without localizing missed prerequisites. In this loop, successful demonstrations initialize reward and policy, history-dependent reward predictions guide PPO, and terminal outcomes trigger updates on recurring high-scoring failures.

Reward learning uses Adam with learning rate 10^{-4} , batch size 8, and 60 epochs. PPO uses learning rate 3×10^{-4} , clip ratio 0.2, discount 0.99, GAE 0.95, 2,048-step rollouts, and ten optimization epochs per update. The interaction budgets are 2×10^5 steps per simulation task and 120 episodes per real-robot task at 10 Hz, with a 500-step horizon. Experiments use seeds 0, 1, and 2 on eight RTX 4090 GPUs.

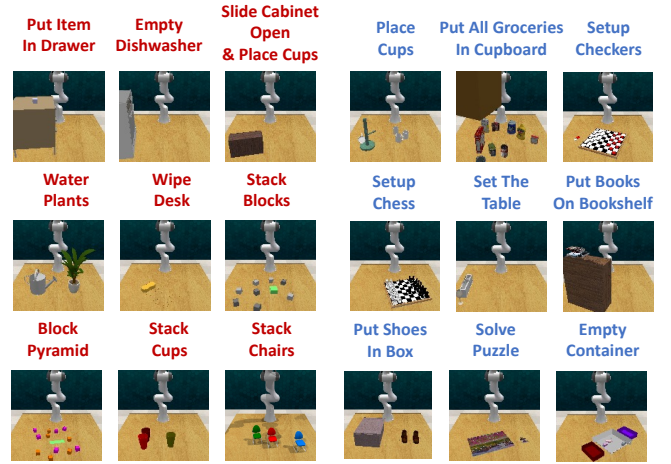


Fig. 3. The 18 long-horizon RL Bench evaluation tasks.

IV. EXPERIMENTS

We examine the causal-consistency objective through synthetic order-perturbation controls and real-execution examples, and downstream utility through task success and ablations. We address four questions: (1) Does RoboLongR improve complete-task success over existing reward-learning approaches on long-horizon manipulation tasks (Sec. IV-B)? (2) Can the learned reward distinguish held-out demonstrations from synthetic trajectories with perturbed stage orders (Sec. IV-C)? (3) How do stage-shuffle supervision, recurrent memory, stage scheduling, and online reward correction contribute to policy performance (Sec. IV-D)? (4) Do task-success gains extend to real robots, and how does the reward respond to representative prerequisite-violating executions (Sec. IV-E)?

A. Experimental Setup

Simulation Benchmark. Our RL Bench [23] protocol separates 88 reward-pretraining tasks from 18 long-horizon evaluation tasks (Fig. 3). The 88-task set supplies 20 successful demonstrations per task, totaling 1,760 trajectories for offline reward pretraining. Each evaluation task separately provides 20 demonstrations for reward relabeling and BC initialization. Subsequent policy learning and online reward correction use target-task interaction, so the split evaluates downstream improvement on tasks held out from reward pretraining. Reward pretraining also includes 4,000 trajectories from the bridge and fractal20220817 subsets of Open-X Embodiment [24], comprising 25% of each batch. Open-X supplies progress supervision and rewound negatives; stage-shuffle uses segmented RL Bench demonstrations. ReWiND is retrained on the same data mixture.

The evaluation tasks comprise nine strong-dependency tasks with prominent physical prerequisites and nine weak-dependency tasks permitting flexible object order. For example, drawer storage requires opening before placement, whereas table setting permits independent object placements. Both groups may contain partial orders: support blocks can be interchangeable, and an otherwise flexible task may require opening a lid. The split tests whether improvements

TABLE I

SUCCESS RATES (%) ON 18 LONG-HORIZON RLBENCH TASKS GROUPED BY DEPENDENCY STRENGTH. RESULTS REPORT MEAN $\pm$ SAMPLE STANDARD DEVIATION OVER THREE SEEDS, WITH 50 TRIALS PER SEED. AVG. DENOTES THE WITHIN-GROUP MEAN.

<i>Strong-dependency tasks with prominent physical prerequisites</i>											
Models	Avg. (%) $\uparrow$	Put Item In Drawer	Empty Dishwasher	Slide Cabinet Open & Place Cups	Water Plants	Wipe Desk	Block Pyramid	Stack Blocks	Stack Cups	Stack Chairs	
BC-only	8.2	14.0 $\pm$ 3.5	9.3 $\pm$ 2.3	6.7 $\pm$ 3.1	16.7 $\pm$ 3.1	20.0 $\pm$ 2.0	0.0	2.7 $\pm$ 1.2	4.7 $\pm$ 4.2	0.0	
Sparse-PPO	7.1	12.7 $\pm$ 2.3	8.0 $\pm$ 3.5	5.3 $\pm$ 2.3	14.7 $\pm$ 4.2	18.0 $\pm$ 3.5	0.0	2.0 $\pm$ 2.0	3.3 $\pm$ 3.1	0.0	
LIV	3.5	6.7 $\pm$ 4.2	4.0 $\pm$ 3.5	1.3 $\pm$ 2.3	8.0 $\pm$ 3.5	11.3 $\pm$ 1.2	0.0	0.0	0.0	0.0	
Progress+Mono	20.9	30.7 $\pm$ 3.1	24.0 $\pm$ 2.0	20.0 $\pm$ 2.0	33.3 $\pm$ 4.2	38.0 $\pm$ 2.0	8.7 $\pm$ 3.1	11.3 $\pm$ 4.2	15.3 $\pm$ 3.1	6.7 $\pm$ 4.2	
ReWiND	23.6	34.0 $\pm$ 3.5	27.3 $\pm$ 2.3	22.7 $\pm$ 4.2	36.7 $\pm$ 4.2	41.3 $\pm$ 1.2	10.0 $\pm$ 2.0	14.0 $\pm$ 2.0	17.3 $\pm$ 4.2	8.7 $\pm$ 2.3	
RoboLongR	37.8	49.3$\pm$2.3	42.0$\pm$2.0	36.7$\pm$2.3	51.3$\pm$2.3	55.3$\pm$3.1	22.7$\pm$3.1	27.3$\pm$4.2	32.0$\pm$3.5	23.3$\pm$4.2	

<i>Weak-dependency tasks allowing flexible subtask order</i>											
Models	Avg. (%) $\uparrow$	Place Cups	Put All Groceries In Cupboard	Setup Checkers	Setup Chess	Set The Table	Put Books On Bookshelf	Put Shoes In Box	Solve Puzzle	Empty Container	
BC-only	7.4	16.0 $\pm$ 4.0	2.0 $\pm$ 3.5	3.3 $\pm$ 1.2	0.0	0.0	12.7 $\pm$ 4.2	10.0 $\pm$ 4.0	0.0	22.7 $\pm$ 4.2	
Sparse-PPO	6.5	14.0 $\pm$ 3.5	1.3 $\pm$ 1.2	2.7 $\pm$ 2.3	0.0	0.0	11.3 $\pm$ 3.1	8.7 $\pm$ 2.3	0.0	20.7 $\pm$ 2.3	
LIV	2.4	6.0 $\pm$ 4.0	0.0	0.0	0.0	0.0	4.7 $\pm$ 1.2	4.0 $\pm$ 3.5	0.0	6.7 $\pm$ 2.3	
Progress+Mono	19.3	30.7 $\pm$ 2.3	10.7 $\pm$ 3.1	13.3 $\pm$ 4.2	6.7 $\pm$ 1.2	5.3 $\pm$ 3.1	26.0 $\pm$ 2.0	22.7 $\pm$ 3.1	7.3 $\pm$ 2.3	50.7 $\pm$ 4.2	
ReWiND	21.6	34.0 $\pm$ 2.0	12.7 $\pm$ 4.2	15.3 $\pm$ 2.3	8.0 $\pm$ 2.0	6.7 $\pm$ 3.1	29.3 $\pm$ 1.2	26.0 $\pm$ 2.0	8.7 $\pm$ 2.3	54.0 $\pm$ 3.5	
RoboLongR	25.9	38.0$\pm$2.0	16.7$\pm$4.2	19.3$\pm$2.3	11.3$\pm$4.2	10.0$\pm$4.0	33.3$\pm$2.3	34.0$\pm$2.0	12.7$\pm$3.1	57.3$\pm$3.1	

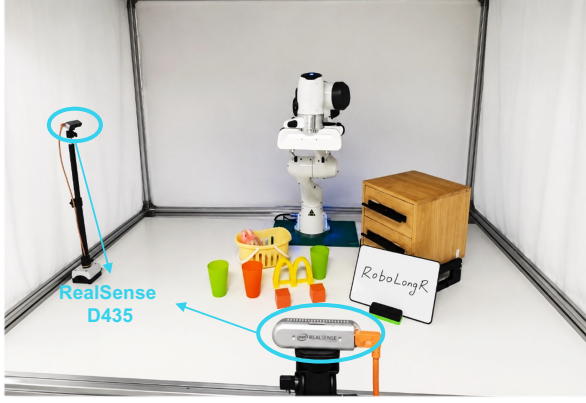


Fig. 4. Real-world robot setup.

are larger when physical dependencies are prominent; it does not isolate their causal mechanism.

Real-Robot Platform. We use a seven-degree-of-freedom Franka Panda with front and right Intel RealSense D435 cameras (Fig. 4). Four tasks (Fig. 5) cover tool use, assembly, multi-object manipulation, and articulated-object interaction. Board Wiping requires retrieving a tool before cleaning; Arch Bridge Construction requires two supports before placing the arch; Tabletop Tidying requires stacking three cups before transferring them into a basket; and Drawer Object Storage requires opening, inserting, and closing. Both methods share ten successful GELLO [25] demonstrations per task for reward learning and BC initialization. Success requires a clean marked region with the tool released, an arch resting on upright supports, a stable cup stack inside the basket, or an object inside the fully closed drawer, respectively.

Baselines and Evaluation. All RL comparisons share demonstrations, stage-conditioned policy architecture, BC initialization, and interaction budgets. BC-only measures

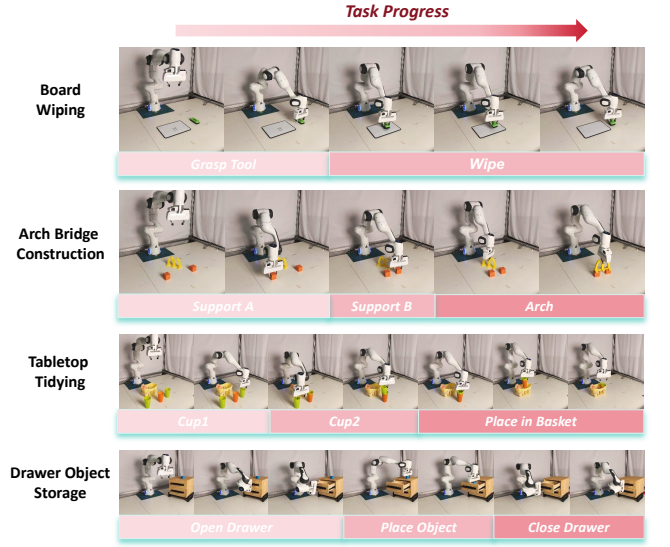


Fig. 5. The four real-robot long-horizon tasks, each shown as a sequence of subtasks.

performance before online refinement. Sparse-PPO uses binary terminal feedback, while LIV [3] supplies a pre-trained visual-language reward without task-specific training. ReWiND [5] is the closest progress-reward baseline and differs in its negative construction. Progress+Mono retains RoboLongR’s reward architecture but replaces stage-shuffle with a stage-prediction head and a penalty on decreasing predicted stage indices. Its loss combines positive progress regression, stage cross-entropy, and an expected-stage monotonicity penalty, with unit weights on both auxiliary terms. LIV scores are min-max normalized within each rollout before PPO updates.

Simulation evaluation uses 50 episodes per task per seed,

TABLE II
REWARD-LEVEL ORDER SENSITIVITY ON HELD-OUT
TRAJECTORIES.

	ReWiND	RoboLongR
$\Delta(\text{order-perturbed}) \uparrow$	0.04	0.51
$\Delta(\text{rewound}) \uparrow$	0.38	0.42
$\Delta(\text{spliced}) \downarrow$	0.03	0.06
Ranking accuracy (%)	54.2	91.7
Progress MSE on correct traj.	0.021	0.023

with randomized layouts and three seeds (150 trials per task). Real-robot evaluation compares RoboLongR with ReWiND, the strongest simulation baseline, using ten trials per task and method with randomized placements. A trial succeeds only when every subtask is completed, judged from video by one annotator blind to the method. Failed trials are not selectively repeated.

B. Main Results on RL Bench

Table I lists task averages of 31.8% for RoboLongR and 22.6% for ReWiND. BC-only, Sparse-PPO, LIV, and Progress+Mono yield 7.8%, 6.8%, 2.9%, and 20.1%, respectively. Computed before rounding, the difference between RoboLongR and ReWiND is 9.2 percentage points. The group averages are 37.8% versus 23.6% on strong-dependency tasks and 25.9% versus 21.6% on weak-dependency tasks for RoboLongR and ReWiND. The differences, computed before rounding, are 14.2 and 4.2 percentage points. RoboLongR exceeds ReWiND on all 18 task means. The larger gain in the strong-dependency group is consistent with the motivation for rewards sensitive to task prerequisites.

The comparison with BC-only shows that the result reflects substantial improvement beyond demonstration initialization. Sparse-PPO remains close to that reference under the same interaction budget, whereas the learned progress baselines achieve higher success. Progress+Mono improves over BC but remains below ReWiND and RoboLongR, suggesting that an architecture-matched stage objective alone is insufficient to reproduce the gains. These comparisons evaluate the complete training pipelines; differences in negative construction and supervision must be considered when attributing the improvement to individual design choices.

Task-level results further clarify the scope of the gain. RoboLongR reaches 49.3% on Put Item In Drawer and 22.7% on Block Pyramid, compared with 34.0% and 10.0% for ReWiND. Both tasks impose physical prerequisites, but the lower absolute success on assembly shows that a better reward does not remove manipulation difficulty. In the weak-dependency group, Setup Chess and Set The Table remain at 11.3% and 10.0%. Thus improvement is broad across the benchmark, while complete-task reliability remains limited on demanding compositions.

C. Order Sensitivity of the Reward

We examine stage-order sensitivity as a diagnostic of the causal-consistency objective, evaluating rewards inde-

pendently of the policy on held-out demonstrations. Correct trajectories preserve the demonstrated order. Order-perturbed trajectories swap intact stage segments. Rewound trajectories use regression-style corruption. Order-preserving splices replace a segment with the corresponding stage from another successful demonstration. We compute $\Delta(x) = \bar{r}(\text{correct}) - \bar{r}(x)$, where $\bar{r}(\tau) = T^{-1} \sum_t R_\phi(\tau, \ell)_t$, and report pairwise ranking accuracy. Progress MSE uses $(t/T)^2$ on correct demonstrations. Training swaps the latter two of three consecutive stages. Evaluation includes pair swaps at different distances, including adjacent and more distant pairs. Thus the evaluation expands the temporal distances but is not uniformly disjoint from the training corruption family. Valid orders already present in held-out weak-dependency demonstrations remain in the correct set. This does not exhaustively test alternative valid orders.

Table II shows that RoboLongR achieves a larger order-perturbation margin (0.51 versus 0.04) and higher ranking accuracy (91.7% versus 54.2%), while maintaining similar progress MSE on correct trajectories. Its small margin on order-preserving splices (0.06) argues against indiscriminately penalizing all spliced trajectories. However, unmatched local boundary statistics remain a possible short-cut, so these results establish discrimination of the tested synthetic perturbations without fully verifying semantic prerequisites. Further analysis shows sensitivity across swap distances, with RoboLongR margins of 0.42, 0.53, and 0.61 for adjacent, medium, and distant swaps, respectively, compared with 0.03, 0.04, and 0.06 for ReWiND.

D. Ablation Studies

Fig. 7 presents component ablations under shared demonstrations and interaction budgets. Removing stage-shuffle reduces the reported success rate from 31.8% to 24.3%. Removing correction, memory, or the stage schedule yields 28.4%, 28.9%, and 29.8%. Stage-shuffle produces the largest observed reduction when removed, while memory, correction, and stage conditioning each contribute within the tested pipeline. The refinement experiments compare one, two, and three reward-training rounds, with reported success rates of 28.8%, 31.8%, and 32.0%. Two rounds capture most of the observed gain. Replacing adaptive shuffle segments with coarse segments yields 29.8%. Using adaptive indices for both reward and policy yields 30.4%. These comparisons favor distinct segmentations for reward augmentation and policy conditioning.

The segmentation analysis also reveals where adaptation is used. Curvature-based boundaries are detected in 91% of opening-and-closing demonstrations, compared with 78% for pick-and-place, 84% for insertion and alignment, and 52% for composition. Their mean adaptive stage counts are 3.8, 4.6, 4.1, and 6.2, respectively. Repeated object manipulations can produce smoother progress curves, so the five-segment fallback remains important. The first-to-second training-round improvement accompanies an increase in reward margin from 0.38 to 0.51; the third round reaches only 0.52. Together, these results motivate two rounds without

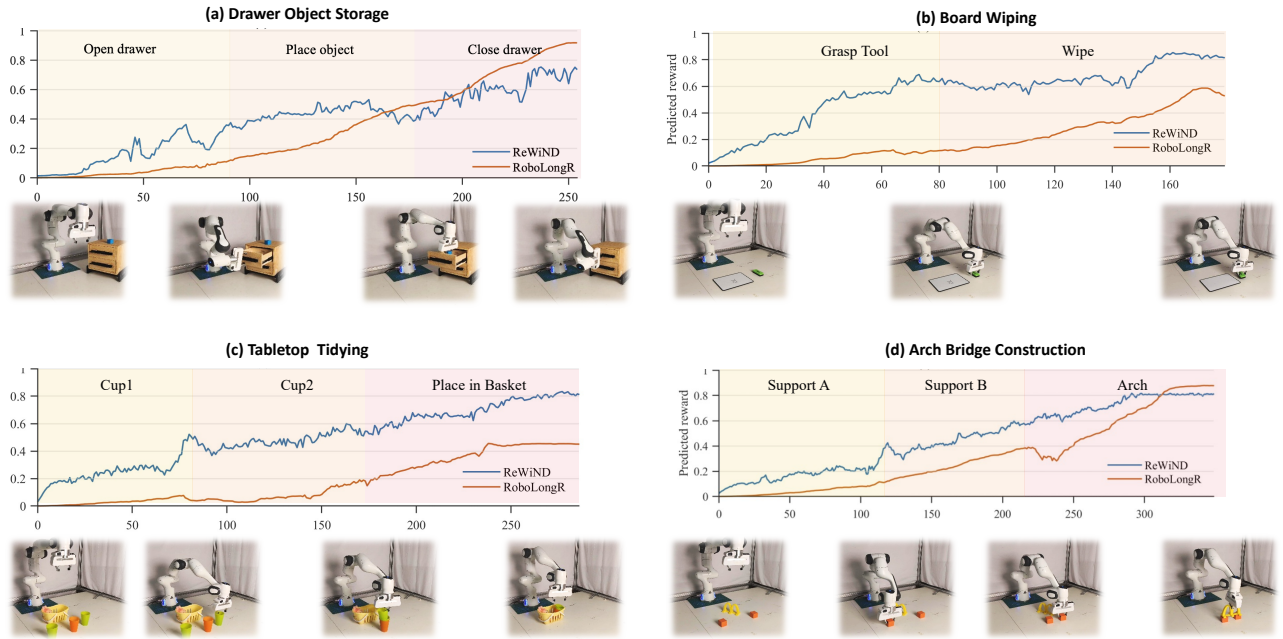


Fig. 6. **Predicted rewards on representative valid executions of the four real-robot tasks.** Shaded regions denote semantic task stages, and the images below each curve show representative observations. Both methods track task progress, although RoboLongR produces lower raw reward values on some tasks.

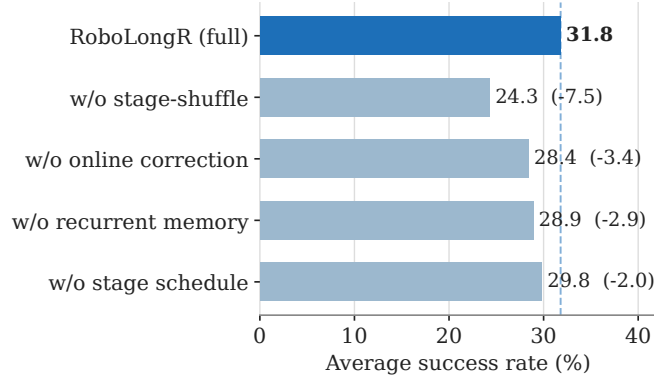


Fig. 7. Ablation of RoboLongR components.

TABLE III

SUCCESS RATE (%) ON FOUR REAL-ROBOT TASKS, TEN TRIALS PER TASK AND METHOD. BRACKETS GIVE WILSON 95% INTERVALS.

Task	ReWiND	RoboLongR
Tabletop Tidying	20 [5.7, 51.0]	50 [23.7, 76.3]
Drawer Object Storage	20 [5.7, 51.0]	70 [39.7, 89.2]
Arch Bridge Construction	10 [1.8, 40.4]	40 [16.8, 68.7]
Board Wiping	50 [23.7, 76.3]	70 [39.7, 89.2]
Pooled (40 trials)	25.0 [14.2, 40.2]	57.5 [42.2, 71.4]

implying that additional self-training will always improve the reward.

E. Real-Robot Validation

Table III reports higher observed success for RoboLongR on all four tasks, with 23/40 successes pooled across tasks compared with 10/40 for ReWiND. The pooled rates are 57.5% and 25.0%, a 32.5-point gain. With ten trials per task,

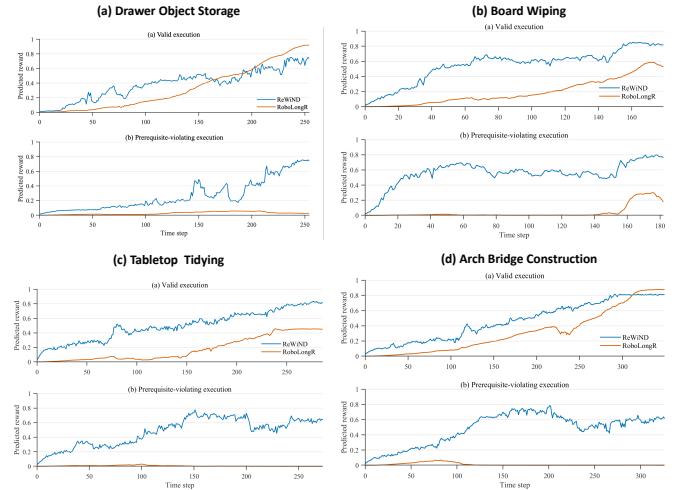


Fig. 8. Predicted rewards on representative valid (upper) and prerequisite-violating (lower) executions of four real-robot tasks. RoboLongR shows clearer separation than ReWiND in these examples.

the Wilson intervals are wide; they summarize this fixed task mixture and do not establish per-task statistical significance.

Reward analysis uses separately recorded held-out rollouts excluded from reward training, BC, policy optimization, and model selection. The drawer pair in Fig. 8 illustrates valid and failed executions. A premature placement attempt alone need not make a trajectory irrecoverable. The relevant negative example is an execution that ultimately fails to store the object. These curves provide qualitative evidence about the sampled rollouts, not a comprehensive recovery test.

Reward Progress on Real Executions. Figure 6 shows reward progress and representative camera observations for valid executions of all four real-robot tasks. Shaded regions

indicate semantic stages and align the curves with the physical sequence of manipulation. Both methods respond to progress, with lower raw RoboLongR scores on some tasks. Because the models are trained independently, absolute score magnitudes are not directly comparable. We therefore also examine the separation between valid and prerequisite-violating executions within each model.

Sensitivity to Prerequisite Violations. Fig. 8 pairs a representative valid execution with a prerequisite-violating execution for each task. ReWiND assigns substantial rewards to both types of execution in these examples. RoboLongR shows greater separation, particularly for Drawer Object Storage, Tabletop Tidying, and Arch Bridge Construction. On Board Wiping, its reward rises modestly near the end of the invalid execution as the scene resembles completed wiping, while remaining below its reward for the valid execution. This residual response illustrates the difference between a learned progress estimate and a symbolic verifier. These sampled curves qualitatively illustrate reward responses to prerequisite violations on real executions, without establishing coverage of all failure modes or isolating the effect of stage-shuffle from the other components.

V. CONCLUSION AND DISCUSSION

We present RoboLongR, an offline-to-online framework for history-dependent reward learning that targets causal consistency. RoboLongR combines stage-order supervision and recurrent memory with stage-conditioned policy refinement and online reward correction. Following reward pretraining on 88 RL Bench tasks, evaluations on 18 long-horizon tasks excluded from pretraining and four real-robot tasks demonstrate improved complete-task success over the tested baselines. Reward-level controls show sensitivity to synthetic order perturbations, while removing stage-shuffle yields the largest success-rate drop among the component ablations.

Several limitations remain. Synthetic shuffles may introduce boundary shortcuts, while their targets can penalize valid alternative orders or recoveries. Whole-rollout correction may suppress valid prefixes and cause reward drift. The framework relies on demonstrations, engineered stage cues, and terminal feedback. Future work will explore localized reward correction, evaluation of valid execution orders and recoveries, and stability over extended policy improvement.

REFERENCES

- [1] A. Pan, K. Bhatia, and J. Steinhardt, "The effects of reward misspecification: Mapping and mitigating misaligned models," in *International Conference on Learning Representations (ICLR)*, 2022.
- [2] J. Skalse, N. H. R. Howe, D. Krashennikov, and D. Krueger, "Defining and characterizing reward hacking," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [3] Y. J. Ma, V. Kumar, A. Zhang, O. Bastani, and D. Jayaraman, "Liv: Language-image representations and rewards for robotic control," in *International Conference on Machine Learning (ICML)*, 2023.
- [4] S. A. Sontakke, J. Zhang, S. M. R. Arnold, K. Pertsch, E. Bryk, D. Sadigh, C. Finn, and L. Itti, "Roboclip: One demonstration is enough to learn robot policies," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [5] J. Zhang, Y. Luo, A. Anwar, S. A. Sontakke, J. J. Lim, J. Thomason, E. Bryk, and J. Zhang, "Rewind: Language-guided rewards teach robot policies without new demonstrations," in *Proceedings of the 9th Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 460–488.
- [6] J. Luo, C. Xu, J. Wu, and S. Levine, "Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning," *arXiv preprint arXiv:2410.21845*, 2025.
- [7] K. Lei, H. Li, D. Yu, Z. Wei, L. Guo, Z. Jiang, Z. Wang, S. Liang, and H. Xu, "RL-100: Performant robotic manipulation with real-world reinforcement learning," *arXiv preprint arXiv:2510.14830*, 2025.
- [8] P. Abbeel and A. Y. Ng, "Apprenticeship learning via inverse reinforcement learning," in *International Conference on Machine Learning (ICML)*, 2004.
- [9] J. Ho and S. Ermon, "Generative adversarial imitation learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [10] M. Alakuijala, R. McLean, I. Woungang, N. Farsad, S. Kaski, P. Marttinen, and K. Yuan, "Video-language critic: Transferable reward functions for language-conditioned robotics," *arXiv preprint arXiv:2405.19988*, 2024.
- [11] Y. J. Ma, J. Hejna, C. Fu, D. Shah, J. Liang, Z. Xu, S. Kirmani, P. Xu, D. Driess, T. Xiao, *et al.*, "Vision language models are in-context value learners," in *International Conference on Learning Representations (ICLR)*, 2025.
- [12] A. Liang, Y. Korkmaz, J. Zhang, M. Hwang, A. Anwar, S. Kaushik, A. Shah, A. S. Huang, L. Zettlemoyer, D. Fox, Y. Xiang, A. Li, A. Bobu, A. Gupta, S. Tu, E. Bryk, and J. Zhang, "Robometer: Scaling general-purpose robotic reward models via trajectory comparisons," in *Robotics: Science and Systems (RSS)*, 2026, arXiv:2603.02115.
- [13] T. Lee, A. Wagenmaker, K. Pertsch, P. Liang, S. Levine, and C. Finn, "Roboreward: General-purpose vision-language reward models for robotics," *arXiv preprint arXiv:2601.00675*, 2026.
- [14] S. Zhai, Q. Zhang, T. Zhang, F. Huang, H. Zhang, M. Zhou, S. Zhang, L. Liu, S. Lin, and J. Pang, "A vision-language-action-critic model for robotic real-world reinforcement learning," *arXiv preprint arXiv:2509.15937*, 2025.
- [15] Q. Chen, J. Yu, M. Schwager, P. Abbeel, Y. Shentu, and P. Wu, "Sarm: Stage-aware reward modeling for long horizon robot manipulation," in *International Conference on Learning Representations (ICLR)*, 2026.
- [16] Q. Chen, H. Zheng, J. Yu, S. Huang, J. Sun, K. Goldberg, C. Wen, P. Abbeel, Y. Shentu, P. Wu, and M. Schwager, "Sarm2: Multi-task stage aware reward modeling for self improving robotic manipulation," *arXiv preprint arXiv:2606.10305*, 2026.
- [17] P. Yang, X. Wang, P. Jing, K. Wen, Y. Qu, Z. Huang, M. Fu, X. Liu, Y. Shen, and F. Shi, "Rarm: Confidence-gated progress reward modeling for rl in manipulation," *arXiv preprint arXiv:2606.22027*, 2026.
- [18] J. Yang, K. Lin, J. Li, W. Zhang, T. Lin, L. Wu, Z. Su, H. Zhao, Y.-Q. Zhang, L. Chen, P. Luo, X. Yue, and H. Li, "Rise: Self-improving robot policy with compositional world model," *arXiv preprint arXiv:2602.11075*, 2026.
- [19] Y. Liu, H. Li, S. Tian, Y. Qin, Y. Chen, Y. Zheng, Y. Huang, and D. Zhao, "Towards long-lived robots: Continual learning via models via reinforcement fine-tuning," *arXiv preprint arXiv:2602.10503*, 2026.
- [20] H. Zhang, S. Zhang, J. Jin, Q. Zeng, R. Li, and D. Wang, "Robustvla: Robustness-aware reinforcement post-training for vision-language-action models," *arXiv preprint arXiv:2511.01331*, 2025.
- [21] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, *et al.*, "Dinov2: Learning robust visual features without supervision," *Transactions on Machine Learning Research (TMLR)*, 2024.
- [22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [23] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, "Rlbench: The robot learning benchmark and learning environment," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3019–3026, 2020.
- [24] A. O'Neill, A. Rehman, A. Gupta, A. Maddukuri, A. Gupta, A. Padalkar, *et al.*, "Open x-embodiment: Robotic learning datasets and RT-X models," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [25] P. Wu, Y. Shentu, Z. Yi, X. Lin, and P. Abbeel, "Gello: A general, low-cost, and intuitive teleoperation framework for robot manipulators," in

2024 *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 12 156–12 163.